Extraction & Parsing

Modulo M2 · Lezione 26



Da martedì a stasera

martedì · L25

Un testo scritto per l'occasione

Un solo esempio bastava a dimostrare il pattern.

stasera · L26

Un documento vero

Più lungo, più sporco — e i campi non ci sono mai tutti.

> Il pattern tecnico (Pydantic + structured outputs) è identico e **non si ri-spiega**: cambia il terreno.



Agenda

Un solo file per tutta la serata: `working/extract.py`.

	Blocco	Dove si lavora	Tempo
📊	Teoria: estrazione vs RAG, casi d'uso, documenti sporchi, campi mancanti, precisione/recall	slide	35′
🎙	Live coding: fattura Lumen, schema con campi nullable, un caso impreciso	`working/extract.py`	65′
⌨	Esercizio breve: un CV dato, il vostro schema	`working/extract.py`	15′
📊	Wrap-up: recap + ponte al progetto	slide	5′

🎙 = scrive il docente, voi replicate ⌨ = scrivete voi

> Chiave OpenAI: quella di **L13–L25**, nessuna registrazione nuova.



1. Estrazione, documenti veri



Estrazione vs RAG

RAG · L19–L24

Una domanda, una collezione

Query → ricerca sulla collezione di documenti → risposta generata dai pezzi trovati.

`query` → `search` → `risposta`

estrazione · stasera

Un documento, un record

Nessuna domanda, nessuna ricerca: un documento dato per intero → un record nella forma X.

`documento` → `record`

> Niente chunking, niente indice, niente query: due strumenti diversi per compiti diversi.



Casi d’uso di automazione

Fatture
Dati contabili verso un gestionale.

CV
Candidati verso un database di selezione.

Email
Richieste verso un ticket strutturato.

Un testo che una persona leggerebbe in pochi secondi diventa un record che un programma processa subito.



Documenti sporchi: va bene lo stesso

Una fattura vera ha intestazione, loghi testuali, footer legale — "rumore" nel senso di L17. L'estrazione lo tollera bene: il modello legge il testo **intero** e capisce cosa è il numero fattura e cosa è rumore.

Non è un invito a non pulire mai nulla: è la ragione per cui stasera **non si ripete** il lavoro di L17. Il rumore che rovina un embedding non rovina allo stesso modo un'estrazione, che lavora sul testo intero, non su similarità vettoriale.



Campi mancanti: nullable sul serio

L'insight di martedì ("aggiungete campi extra, purché nullable") oggi è la normalità dei documenti veri.

```
sconto: float | None = None
```

Un campo **obbligatorio** su un documento che non lo contiene forza il modello a inventare il valore: lo schema resta valido, il dato no. `nullable` evita l'invenzione.



Precisione e recall, per la prima volta

Tre esiti possibili confrontando l'estratto col documento vero:

TP	FP	FN
dato vero, estratto correttamente	dato estratto ma sbagliato o inventato	dato vero, non estratto

```
precision = TP / (TP + FP)
```

```
recall = TP / (TP + FN)
```

Recall basso

Molti `FN` : righe reali non estratte, il record ha meno informazione del documento.

Precisione bassa

Molti `FP` : dati sbagliati o inventati — sembrano corretti, ma non lo sono.

Un buon **prompt** (le strategie di L23) resta la prima difesa contro entrambi — qui lo diamo per acquisito.



2. AI codice



Live coding

Si guarda e si replica, in quattro tappe

`working/extract.py · 65 minuti`

si torna alle slide
per il wrap-up

`Tappa 1 · ~15'`

Carico la fattura

Testo già pulito: lettura diretta del file.

`Tappa 2 · ~25'`

Schema ed estrazione

Pattern di L25, campi nullable veri.

`Tappa 3 · ~15'`

Il nullable in azione

Una seconda fattura, senza sconto.

`Tappa 4 · ~10'`

Un caso impreciso

Precisione e recall, dal vivo.



AL RIENTRO

Tappa 1 — carico la fattura

```
testo = open("dataset/fattura_2026-0143.txt").read()
```

La fattura è già testo pulito: basta una lettura diretta. Per un PDF si riuserebbe `aikit.load()` di L16 — qui non serve.

```
LUMEN S.r.l. — Via dell'Artigianato 14, 40100 Bologna — P.IVA 03214560371
    FATTURA N. 2026-0143 del 14/07/2026
Cliente: Bianchi Arredamenti S.r.l.
...
    Documento generato automaticamente — validità fiscale ai sensi dell'art. 21 DPR 633/72
```

> Intestazione e footer legale ben visibili — **nessun cleaning** prima di procedere.



AL RIENTRO

Tappa 2 — schema ed estrazione

```python
class RigaFattura(BaseModel):
    prodotto: str
    quantita: int
    prezzo_unitario: float

class Fattura(BaseModel):
    numero: str
    data: str
    cliente: str
    righe: list[RigaFattura]
    imponibile: float
    iva: float
    sconto: float | None = None
    totale: float
    note: str | None = None
```

```
numero='2026-0143' · imponibile=1840.00 · iva=404.80 · sconto=92.00 · totale=2152.80
```

1840,00 + 404,80 − 92,00 = **2152,80** — i numeri sono stati letti bene, non inventati.



AL RIENTRO

Tappa 3 — il nullable in azione

Fattura 2026-0158, senza sconto e senza nota.

```
Fattura 2026-0143
sconto=92.00
note='Consegna prevista in 5 giorni lavorativi.'
```

```
Fattura 2026-0158
sconto=None
note=None
```

Stesso schema, stesso codice: il campo assente torna `None`, l'estrazione non fallisce e non inventa un numero.



AL RIENTRO

Tappa 4 — un caso impreciso

"In aggiunta, in omaggio per il cliente: un supporto da scrivania per monitor (nessun addebito)." — scritto in prosa, non come riga "Nx prodotto @ prezzo".

Recall basso, riprodotto

Il supporto monitor NON compare tra le righe estratte: solo la scrivania. Un dato reale è sparito — perché non segue il pattern testuale delle righe vere, il modello non lo riconosce come una riga.

> Stesso principio, direzione opposta: un omaggio scritto *come* una riga (es. "1x supporto monitor @ 0,00 EUR") il modello lo prende — qui non lo era, e infatti manca.



Esercizio breve

TOCCA A VOI

`working/extract.py` · 15 minuti

si torna alle slide
per il wrap-up

«Sono Marta Colombo, sviluppatrice software con sei anni di esperienza. Ho lavorato due anni come frontend developer presso WebNova, poi quattro anni come full-stack developer presso DataForge, dove mi occupo di Python e React. Laurea in Informatica all'Università di Bologna. Parlo italiano e inglese fluentemente.»

Scrivete il vostro schema Pydantic (nome, esperienze, competenze) con almeno un campo che nel CV potrebbe mancare — es. le certificazioni — dichiaratelo `nullable` e fatelo popolare dal modello.



AL RIENTRO

Fatto quando

Il record ha i campi giusti popolati, e il campo eventualmente assente nel CV è `None` — non un errore, non un valore inventato per riempire il vuoto.



Dalla forma al documento vero

Martedì la forma esatta, stasera il documento vero: campi mancanti e precisione/recall sono le due preoccupazioni in più.

Il modulo `working/extract.py` resta come pattern riusabile, insieme a quello di L25.



Ponte: sabato, il Progetto Modulo 2

L'output strutturato entra nella pipeline finale **dove serve** — può essere l'ultimo passo dopo un RAG, o un modulo a sé, a seconda del progetto di ciascuno. Tutto il modulo confluisce in un'unica applicazione.



Grazie

Sabato — Progetto Modulo 2

